ExamGuard AI — System Overview

ExamGuard AI — Implementation Guide

What is ExamGuard AI?

The Problem We're Solving

Picture this real scenario:

- An exam hall with 100+ students sitting in rows
- Only 2 invigilators walking around
- The exam lasts 3 hours
- Students are creative: tiny chits, phone under paper, whispering, looking at neighbor's paper

Can 2 humans watch 100 students for 3 hours straight?

No. It's physically impossible. Invigilators get tired, they can only look in one direction at a time, and they miss things. Studies show that human invigilators catch less than 30% of cheating attempts.

The Solution: ExamGuard AI

ExamGuard is an AI-powered exam monitoring system that watches ALL cameras simultaneously and flags suspicious behavior to human invigilators.

Think of it like this:

Without ExamGuard: 2 tired humans trying to watch 100 students

With ExamGuard: AI watching every student every second + 2 humans making final decisions

The AI doesn't punish anyone. It doesn't decide who cheated. It simply says:

"Hey invigilator, Student in Row 3, Seat 7 has been looking at their neighbor's paper for 15 seconds. You might want to check."

How It Works (The Pipeline)

Here's the complete flow from camera to alert:

```
Step 1: CAMERAS capture video
   |
   v
Step 2: AI PROCESSES each frame
   |
   v
Step 3: AI DETECTS suspicious behavior
   |
   v
Step 4: AI ALERTS the invigilator
   |
   v
Step 5: HUMAN DECIDES what to do
```

Step-by-Step Breakdown:

Step 1 - Video Input:

Cameras in the exam hall continuously capture video. Each camera covers a section of students.

Step 2 - AI Processing:

Every frame (image) from every camera is sent to the AI system. The AI looks at each frame and asks: "What's happening here?"

Step 3 - Detection:

The AI looks for specific things:

- Is someone looking at another's paper?
- Is there a phone visible?
- Is someone passing notes?
- Is behavior unusual compared to normal exam-taking?

Step 4 - Alert:

If the AI is suspicious enough (above a confidence threshold), it sends an alert to the invigilator's dashboard with:

- Which student (location in the hall)
- What behavior was detected
- A short video clip of the incident
- Confidence level (how sure the AI is)

Step 5 - Human Decision:

The invigilator looks at the alert, watches the clip, and decides:

- False alarm? Dismiss it.
- Looks suspicious? Go check in person.
- Definite cheating? Take action per university rules.

Core Capabilities

Here's what ExamGuard can detect and how:

Capability	What It Does	AI Technology	Example
Face Detection	Finds and identifies every student	YOLO object detection	Locate all 100 faces in the frame
Gaze Tracking	Where are they looking?	CNN (Convolutional Neural Network)	Student looking left for 10+ seconds
Object Detection	Spots prohibited items	YOLO	Phone detected under exam paper
Behavior Analysis	Understands what movements mean	CNN + LSTM (sequence model)	Repeated head turning pattern
Anomaly Detection	Finds anything unusual	Autoencoder (unsupervised)	Student suddenly very still (hiding something?)
Alert Decision	When to alert vs ignore	Reinforcement Learning	Don't alert for every head turn, but alert for patterns
Multi-Camera Tracking	Same student across cameras	Person Re-Identification	Track Student #47 from Camera 1 to Camera 3

The Golden Rule: AI Assists, Humans Decide

This is the most important principle of ExamGuard:

```
+-------------------------------------------------+
|                                                 |
|  AI is the DETECTOR, not the JUDGE              |
|                                                 |
|  - AI watches  -->  Human decides               |
|  - AI flags    -->  Human investigates          |
|  - AI suggests -->  Human takes action          |
|                                                 |
|  NO student is ever punished by AI alone.       |
|                                                 |
+-------------------------------------------------+
```

Why this matters:

- 1. AI makes mistakes - It might think scratching your head is suspicious
- 2. Context matters - A student looking around might just be thinking
- 3. Fairness - A human must review before any accusation
- 4. Legal/ethical - Automated punishment without human review is unacceptable

What ExamGuard is NOT

- It is NOT a replacement for invigilators (it's a tool FOR them)
- It does NOT record for punishment (it flags in real-time)
- It does NOT use facial recognition to identify WHO the student is (it tracks positions, not identities)
- It does NOT make academic decisions

Why This Project Matters for Your Learning

Building ExamGuard will teach you:

- 1. Computer Vision - Teaching AI to see and understand images
- 2. Deep Learning - Building neural networks that learn patterns
- 3. Real-time Processing - Handling live video streams
- 4. System Design - Building a complete, working AI system
- 5. Ethics in AI - Balancing surveillance with fairness

By the end, you won't just know AI theory. You'll have built a complete, real-world AI system from scratch.

Quick Summary

Question	Answer
What is ExamGuard?	AI-powered exam cheating detection system
What does it detect?	Phones, wandering eyes, passing notes, unusual behavior
Who makes decisions?	Human invigilators (AI only flags)
What AI is used?	YOLO, CNN, LSTM, Autoencoder, Reinforcement Learning
Is it a camera system?	It uses existing cameras + AI software
Does it replace humans?	No, it helps humans do their job better

ExamGuard System Architecture

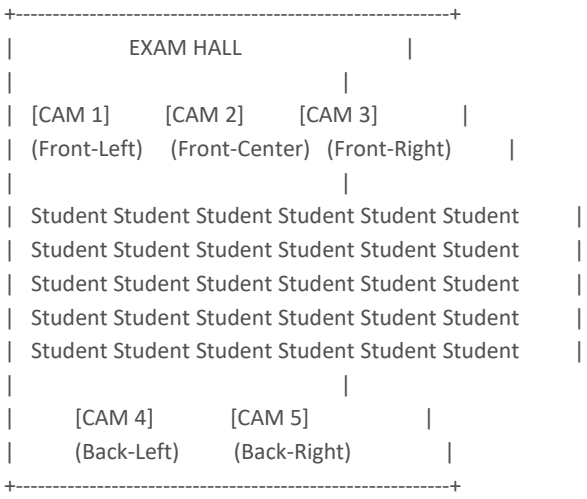
What is System Architecture?

System architecture is the blueprint of how all the parts of a system connect together. Just like a building has a blueprint showing where walls, doors, and wires go, ExamGuard has an architecture showing how cameras, AI models, and dashboards connect.

Before writing a single line of code, you need to understand the big picture.

Hardware Setup

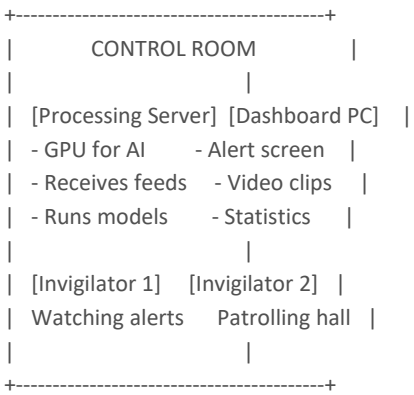
Exam Hall Cameras



Why 5 cameras per hall?

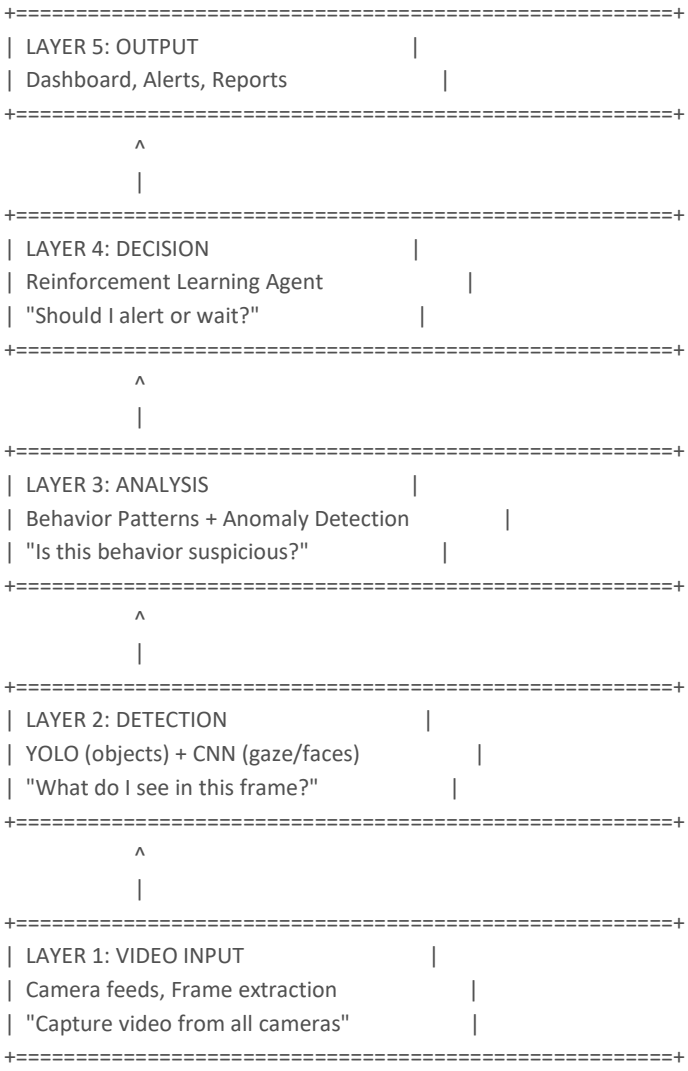
- 3 front cameras: Cover the faces and upper body of students (gaze detection needs to see eyes)
- 2 back cameras: Cover the desks and hands (object detection needs to see what's on the desk)
- Overlapping coverage ensures no blind spots

Control Room



Software Architecture: The 5 Layers

ExamGuard's software is organized into 5 layers, like floors of a building. Each layer has a specific job, and data flows from Layer 1 down to Layer 5.



Layer-by-Layer Explanation

Layer 1: Video Input

What it does: Captures live video from all cameras and converts it into individual frames (images) that AI can process.

Think of it like: Your eyes - they take in raw light and send images to your brain.

Technical details:

- Each camera produces 30 frames per second (fps)
- Each frame is an image (e.g., 1920x1080 pixels)
- Frames are stored as NumPy arrays (matrices of numbers)
- Multiple camera feeds are handled simultaneously

ExamGuard specifics:

- 5 cameras per hall, each at 30 fps = 150 frames per second per hall
- Smart FPS: Only process 5-10 fps normally, jump to 30 fps when something looks suspicious
- Frames are resized to 416x416 or 640x640 for the AI models

Camera Feed → Decode Video → Extract Frames → Resize → Send to Layer 2

Layer 2: Detection (YOLO + CNN)

What it does: Looks at each frame and identifies WHAT is in it. This layer answers: "What objects and people do I see?"

Think of it like: Your brain recognizing faces, objects, and body positions in a photo.

Models used:

Model	Job	Output
YOLOv8	Find objects (phones, chits, books)	Bounding boxes + labels
CNN (face)	Detect and locate faces	Face positions
CNN (gaze)	Determine eye/head direction	Gaze angle (left, right, down, etc.)
Pose Estimator	Detect body posture	Skeleton points (shoulders, elbows, wrists)

ExamGuard specifics:

- YOLO scans each frame for prohibited items (phone, paper chit, earpiece)
- Face CNN locates every student's face and tracks their gaze direction
- Pose estimation sees if someone is leaning toward a neighbor

Frame → YOLO (find objects) → CNN (analyze faces/gaze) → Pose (body position) → Send to Layer 3

Layer 3: Analysis (Behavior + Anomaly)

What it does: Takes the detections from Layer 2 and asks: "Is this behavior suspicious?" This layer looks at PATTERNS over time, not just single frames.

Think of it like: The difference between seeing someone look left once (normal) vs. looking left 15 times in 2 minutes (suspicious).

Models used:

Model	Job	How
CNN + LSTM	Analyze behavior sequences	Watches patterns over 30-60 seconds
Autoencoder	Find anomalies	Learns what "normal" looks like, flags anything different
K-Means Clustering	Group similar behaviors	Finds patterns in student movements

ExamGuard specifics:

- LSTM (Long Short-Term Memory) remembers what happened in previous frames
- A single head turn = normal. Repeated head turns toward same student = suspicious
- Autoencoder learns "normal exam behavior" and flags anything it hasn't seen before
- K-Means groups students by behavior patterns (cluster of students all looking same direction?)

Detections over time → LSTM (sequence patterns) → Autoencoder (anomaly check) → Suspicion score → Send to Layer 4

Layer 4: Decision (Reinforcement Learning)

What it does: Decides WHETHER and WHEN to alert the invigilator. Not every suspicious detection should trigger an alert.

Think of it like: A smart filter that learns from feedback. If it sends too many false alarms, invigilators start ignoring alerts. If it misses real cheating, it's useless.

Model used:

- Deep Q-Network (DQN) or PPO - Reinforcement Learning agent

The balance:

Too many alerts (false alarms) → Invigilator ignores all alerts → System becomes useless
Too few alerts (missed cheating) → Cheating goes undetected → System is useless

GOAL: Just the right amount of HIGH-QUALITY alerts

ExamGuard specifics:

- The RL agent receives a "suspicion score" from Layer 3
- It decides: Alert now? Wait for more evidence? Ignore?
- It learns from invigilator feedback (was the alert helpful or not?)
- Over time, it gets better at knowing when to alert

Suspicion score → RL Agent (alert decision) → If yes → Send to Layer 5

Layer 5: Output (Dashboard + Alerts)

What it does: Shows everything to the invigilator in a clear, easy-to-use interface.

Think of it like: The screen and speakers in a car - they show you the information from all the sensors.

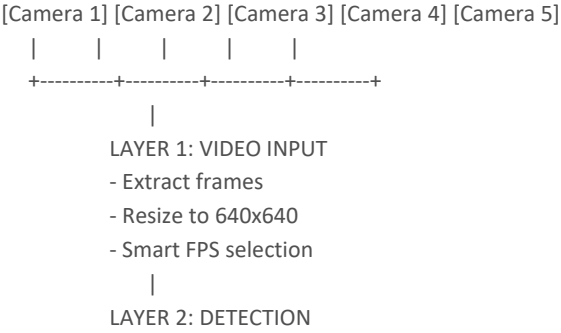
Components:

Component	What It Shows
Live Dashboard	All camera feeds with colored overlays (green = normal, yellow = watch, red = alert)
Alert Panel	New alerts with video clips, confidence level, and description
History Log	All alerts from the current exam session
Statistics	How many alerts, which areas are most active, patterns
Feedback Buttons	"Correct alert" / "False alarm" (trains the RL agent)

Alert data → Dashboard (visual display) → Invigilator sees and acts → Feedback → Back to Layer 4

Complete Data Flow Diagram

Here's how data flows through the entire system from start to finish:



```
- YOLO: "Phone detected!" (85% confidence)
- CNN: "Face looking left" (72% confidence)
- Pose: "Leaning toward neighbor"
      |
LAYER 3: ANALYSIS
- LSTM: "Looking left 8 times in 2 min"
- Autoencoder: "Unusual stillness detected"
- Suspicion score: 0.78 / 1.00
      |
LAYER 4: DECISION
- RL Agent: "Score 0.78 > threshold 0.70"
- Decision: SEND ALERT
      |
LAYER 5: OUTPUT
+---------------------------------+
| ALERT: Row 3, Seat 7            |
| Behavior: Repeated gaze left    |
| Confidence: 78%                 |
| [View Clip] [Dismiss] [Confirm] |
+---------------------------------+
      |
INVIGILATOR DECIDES
- Clicks "Confirm" → walks to check
- Feedback sent to Layer 4 for learning
```

Why This Architecture?

Why layers?

- Each layer has ONE clear job
- You can improve one layer without breaking others
- Easy to test each part separately
- Easy to understand and explain

Why these specific models?

- YOLO is the fastest object detector (real-time!)
- CNN is the standard for image analysis
- LSTM handles sequences (video = sequence of frames)
- Autoencoder is perfect for "find the unusual thing"
- RL learns from feedback (gets better over time)

Why not one big model?

- One model can't do everything well
- Separate models = each specialized for its task
- If one model fails, others still work
- Easier to train and debug

What You'll Build

You won't build all 5 layers at once. Here's the order:

- 1. Phase 1: Foundation knowledge (DONE)
- 2. Phase 2: Python + Libraries (build tools)
- 3. Phase 3: Core ML (learn to train models)
- 4. Phase 4: Computer Vision + YOLO (Layer 2)
- 5. Phase 5: Deep Learning + LSTM (Layer 3)
- 6. Phase 6: Reinforcement Learning (Layer 4)
- 7. Phase 7: Integration (connect all layers)

Each phase builds on the previous one. By the end, you'll have the complete system.

Multi-Camera Handling in ExamGuard

The Challenge: Scale

Let's do the math for a real university exam:

50 exam halls
x 4-5 cameras per hall
= 200+ cameras

Each camera at 30 fps
= 200 x 30 = 6,000 frames per second

Each frame is 1920x1080 pixels
= 6,000 x 6.2 MB = ~37 GB per second of raw data

That's 37 gigabytes of data EVERY SECOND.

No single computer can process all of this in real-time. We need smart strategies.

Solution 1: Edge Processing

What is Edge Processing?

Instead of sending ALL video to one central server, we process video RIGHT AT THE CAMERA (or near it) and only send the interesting parts.

WITHOUT Edge Processing:
[Camera] ---(full video stream)---> [Central Server]
= Massive bandwidth, server overloaded

WITH Edge Processing:
[Camera + Small AI Chip] ---(only suspicious clips)---> [Central Server]
= 90% less data, server handles easily

How it works for ExamGuard:

- 1. A small AI chip (like NVIDIA Jetson) sits next to each camera
- 2. It runs a lightweight YOLO model locally
- 3. It quickly checks: "Is anything suspicious happening?"
- 4. If NO: Send nothing (or just a thumbnail every 5 seconds)
- 5. If YES: Send the full-quality clip to the central server for detailed analysis

The result:

Before edge processing: 6,000 fps to process centrally
After edge processing: ~600 fps to process centrally (90% reduction!)

Most of the time, students are just writing their exam. Nothing to send. Only when something looks off does the full video get analyzed.

Solution 2: Person Re-Identification (Re-ID)

The Problem

A student might appear on Camera 1 (front view) and Camera 4 (side view). The AI sees two different images. How does it know it's the SAME student?

```
Camera 1 sees: [Face from front, blue shirt]
Camera 4 sees: [Side profile, blue shirt]

Are these the same person? The AI needs to figure this out.
```

What is Person Re-ID?

Person Re-Identification is a special AI model that creates a unique "signature" for each person based on their appearance (clothing, body shape, hair) so they can be recognized across different cameras.

Why it matters for ExamGuard:

- Tracking behavior across views: If Camera 1 sees Student A looking suspicious, and Camera 4 has a better angle, the system needs to know it's the SAME student
- Complete behavior picture: Combine observations from multiple cameras for more accurate detection
- Avoid double alerts: Don't alert twice for the same incident seen by two cameras

How it works:

```
Camera 1 frame → Re-ID Model → Feature vector [0.23, 0.87, 0.12, ...]
Camera 4 frame → Re-ID Model → Feature vector [0.25, 0.85, 0.14, ...]

Compare vectors → Very similar! → Same person!
```

The model converts each person's appearance into a list of numbers (feature vector). Similar people have similar vectors.

Solution 3: Smart FPS (Adaptive Frame Rate)

The Idea

Why process 30 frames per second when nothing is happening? Save processing power for when it matters.

```
NORMAL MODE (nothing suspicious):
- Process 5-10 frames per second
- Quick scan for any changes
- Low CPU/GPU usage

ALERT MODE (something detected):
- Jump to 25-30 frames per second
- Detailed analysis of every frame
- Full model pipeline activated

REVIEW MODE (incident being reviewed):
- Record at full 30 fps
```

- Save high-quality clip for invigilator
- Keep processing for 30 seconds after incident

ExamGuard implementation:

State	FPS	Processing	Trigger
Idle	5 fps	Basic motion detection only	Default state
Watch	10 fps	YOLO + face detection	Any motion detected
Suspicious	20 fps	Full detection pipeline	Object or gaze anomaly
Alert	30 fps	All models + recording	High suspicion score

Why this saves resources:

Without Smart FPS: 200 cameras x 30 fps = 6,000 fps to process
With Smart FPS: 200 cameras x avg 8 fps = 1,600 fps to process

That's a 73% reduction in processing load!

At any given moment, most cameras are in "Idle" or "Watch" mode. Only a few cameras with active suspicion need full processing.

Solution 4: GPU Parallel Processing

What is Parallel Processing?

A GPU (Graphics Processing Unit) can process many things at the same time. While a CPU processes tasks one by one, a GPU can handle hundreds simultaneously.

CPU (one task at a time):
Frame 1 → Process → Done
Frame 2 → Process → Done
Frame 3 → Process → Done
Total: 3 units of time

GPU (many tasks at once):
Frame 1 → Process ↘
Frame 2 → Process → All done!
Frame 3 → Process ↗
Total: 1 unit of time

How ExamGuard uses GPUs:

One GPU can handle 20-30 camera feeds simultaneously by:

- 1. Batching: Collect frames from 20 cameras, process them all at once
- 2. Model sharing: One copy of YOLO in GPU memory processes all frames
- 3. Pipeline overlap: While GPU processes batch 1, CPU prepares batch 2

GPU processes: [Batch 1: Cams 1-20] [Batch 2: Cams 21-40] [Batch 3: Cams 41-60]
CPU prepares: [Batch 2 ready] [Batch 3 ready] [Batch 4 ready]

They work in parallel = no waiting!

GPU requirements by scale:

Scale	Cameras	Recommended GPU	Number of GPUs

Small (1-2 halls)	5-10	NVIDIA RTX 3060	1
Medium (5-10 halls)	25-50	NVIDIA RTX 4090	2
Large (10-20 halls)	50-100	NVIDIA A100	2-4
University-wide (50+ halls)	200+	NVIDIA A100 cluster	8+

Scaling Table: How ExamGuard Grows

	Small Setup	**Medium Setup**	**Large Setup**
Exam Halls	1-2	5-10	50+
Cameras	5-10	25-50	200+
Students	50-100	250-500	2,500+
Raw FPS	150-300	750-1,500	6,000+
With Smart FPS	40-80	200-400	1,600+
With Edge Processing	Not needed	60-120	160-400
GPUs Needed	1 (consumer)	1-2 (prosumer)	4-8 (server)
Processing Server	1 desktop PC	1 workstation	Server cluster
Dashboard	1 monitor	2-3 monitors	Web dashboard
Edge Devices	None	Optional	Required
Budget Estimate	$2,000-5,000	$10,000-25,000	$50,000+

For Your Project: Start Small

You don't need to handle 200 cameras to learn the concepts. Here's your development path:

Step 1: ONE webcam feed (your laptop camera)
- Learn video processing basics
- Get YOLO working on live video

Step 2: TWO video files simultaneously
- Learn multi-stream handling
- Basic parallel processing

Step 3: FIVE pre-recorded exam videos
- Simulate a full exam hall
- Test Person Re-ID

Step 4: Scale up with optimization
- Add Smart FPS
- Add GPU batching
- Profile performance

Mini Project Idea:

Build a multi-video processor:

- 1. Open 4 video files at the same time
- 2. Run a simple face detector on each
- 3. Display all 4 with detection boxes
- 4. Measure: How many FPS can you achieve?
- 5. Add batching: Does FPS improve?

This teaches you the fundamentals of multi-camera handling without needing actual exam hall cameras.

Key Takeaways

Strategy	What It Does	Savings
Edge Processing	Process at camera, send only suspicious	90% bandwidth reduction
Person Re-ID	Recognize same person across cameras	Avoid duplicate alerts
Smart FPS	Lower frame rate when nothing happens	73% processing reduction
GPU Batching	Process many frames simultaneously	1 GPU = 20-30 feeds

Combined, these strategies turn an impossible problem (37 GB/sec of data) into a manageable one that runs on affordable hardware.

ML Types in ExamGuard

The Three Types of Machine Learning

ExamGuard uses ALL three types of machine learning, each for a different purpose. Think of them as three different kinds of employees:

SUPERVISED LEARNING = The trained expert (knows exactly what cheating looks like)
UNSUPERVISED LEARNING = The watchful observer (notices anything unusual)
REINFORCEMENT LEARNING = The experienced guard (learns when to raise the alarm)

Type 1: Supervised Learning

What it does in ExamGuard: Answers "WHAT is happening?"

Supervised learning is like training a new invigilator by showing them examples:

"Here's a video of a student using a phone. THIS is cheating."

"Here's a video of a student writing normally. THIS is NOT cheating."

After 10,000 examples, the AI learns to tell the difference.

How it works:

TRAINING:
Input: Video clip of a student
Label: "cheating" or "not cheating" (human-provided)

Show 10,000+ labeled clips to the model
Model learns: "When I see THESE patterns, it's cheating"

AFTER TRAINING:
Input: NEW video clip (never seen before)
Output: "This looks like cheating (87% confident)"

Where ExamGuard uses it:

Task	Input	Label	Model
Phone detection	Video frame	"phone" / "no phone"	YOLOv8
Gaze direction	Face image	"looking left" / "looking at paper" / "looking right"	CNN
Behavior classification	30-second clip	"cheating" / "normal" / "suspicious"	CNN + LSTM
Face detection	Video frame	Face bounding boxes	CNN

Why Supervised for these tasks?

Because we KNOW what cheating looks like. We can label examples. When you have clear categories and labeled data, supervised learning is the best choice.

Training data needed:

Minimum viable: 1,000 labeled clips (basic proof of concept)
Good performance: 5,000 labeled clips (reasonable accuracy)
Production quality: 10,000+ labeled clips (reliable in real exams)

For each clip, a human must label:

- What behavior is shown
- Start and end time of the behavior
- Confidence level (obvious cheating vs borderline)

Type 2: Unsupervised Learning

What it does in ExamGuard: Answers "What PATTERNS are unusual?"

Unsupervised learning doesn't know what cheating looks like. Instead, it learns what NORMAL looks like, and then flags anything that's different.

Think of it this way: You don't need to know what every disease looks like to know someone is sick. You just need to know what "healthy" looks like, and anything that deviates is worth checking.

How it works:

TRAINING:
Input: Thousands of clips of NORMAL exam behavior
No labels needed! Just normal behavior.

Model learns: "Normal looks like THIS"

AFTER TRAINING:
Input: New video clip
Model thinks: "Hmm, this doesn't look like normal... flagging it"

The model doesn't know WHAT is wrong, just that SOMETHING is different.

Where ExamGuard uses it:

Task	Model	What It Catches
Anomaly detection	Autoencoder	Any behavior that's "weird" compared to normal
Behavior clustering	K-Means	Groups of students with similar suspicious behavior

Why Unsupervised for these tasks?

Because we CAN'T label everything! There are cheating methods we haven't even thought of yet. Unsupervised learning catches the unexpected.

Real example:

- Supervised model knows: phone, chit, looking at neighbor
- But what about: Morse code tapping? Specific pen arrangements as signals? Coded coughs?
- Unsupervised catches: "I don't know WHAT this is, but it's not normal exam behavior"

How the Autoencoder works (simplified):

Normal frame → [Compress] → Small representation → [Decompress] → Reconstructed frame

If the reconstruction is GOOD → "I've seen this before" → Normal
If the reconstruction is BAD → "I've never seen this" → Anomaly!

The model learns to compress and reconstruct normal behavior.
When it sees something abnormal, it can't reconstruct it well = FLAGGED.

Clustering example:

K-Means looks at all student behaviors and groups them:

Cluster 1: Students writing steadily → Normal (95% of students)
Cluster 2: Students looking around occasionally → Normal (4% of students)
Cluster 3: Students with very unusual movement → Investigate! (1% of students)

Type 3: Reinforcement Learning

What it does in ExamGuard: Answers "WHEN should I alert?"

Reinforcement Learning (RL) is like training a guard dog. It learns from rewards and punishments.

Every time it makes a correct alert: REWARD (+100 points)

Every time it sends a false alarm: PENALTY (-50 points)

Every time it misses real cheating: BIG PENALTY (-200 points)

Over time, it learns the perfect balance.

Why this is critical:

Scenario A: Alert for EVERYTHING
- Invigilator gets 500 alerts per exam
- 490 are false alarms
- Invigilator starts ignoring ALL alerts
- System becomes USELESS

Scenario B: Alert for NOTHING
- Invigilator gets 0 alerts
- 10 real cheating incidents missed
- System is USELESS

Scenario C: Smart alerts (RL)
- Invigilator gets 15 alerts per exam
- 12 are real cheating (80% precision)
- Invigilator trusts and acts on alerts
- System is VALUABLE

The Reward System:

Action	Result	Points	Why
Alert sent	Invigilator confirms real cheating	+100	Correct alert, system is useful
Alert sent	Invigilator dismisses (false alarm)	-50	Wasted invigilator's time
No alert	Nothing was happening	+10	Correctly stayed quiet
No alert	Cheating was happening (missed!)	-200	The worst outcome!

Why is missing cheating punished MORE than false alarms?

- A false alarm wastes 10 seconds of the invigilator's time
- Missing real cheating defeats the entire purpose of the system
- So the AI learns: "When in doubt, it's better to alert than to miss"
- But not TOO much, or it alerts for everything

How the RL agent learns:

Exam 1: Agent sends 200 alerts, 180 are false alarms
Score: 20 x 100 + 180 x (-50) = -7,000 (terrible!)
Agent learns: "I'm alerting too much"

Exam 2: Agent sends 50 alerts, 30 are false alarms
Score: 20 x 100 + 30 x (-50) = 500 (better!)
Agent learns: "Getting closer"

Exam 10: Agent sends 18 alerts, 3 are false alarms
Score: 15 x 100 + 3 x (-50) = 1,350 (great!)
Agent has learned the right balance

Where ExamGuard uses it:

Task	What It Decides	Model
Alert timing	Alert now or wait for more evidence?	Deep Q-Network (DQN)
Confidence threshold	How suspicious before alerting?	Policy Gradient
Camera priority	Which camera needs attention most?	Multi-Agent RL

How All Three Types Work Together

EXAM IN PROGRESS:

Frame captured from Camera 3
|
v
SUPERVISED (YOLO + CNN):
"I see a phone shape under the paper (73% confident)"
"Student's gaze is toward neighbor (68% confident)"
|
v
UNSUPERVISED (Autoencoder):
"This student's movement pattern is unusual (anomaly score: 0.82)"
"Doesn't match any normal behavior cluster"
|
v
REINFORCEMENT LEARNING (DQN):
"Phone detection: 73% - that's moderate"
"Gaze: 68% - could be just thinking"
"Anomaly: 0.82 - that's high"
"Combined evidence is strong enough"
DECISION: SEND ALERT
|
v
INVIGILATOR receives alert with video clip

Summary: Which Type for What?

Type	Question It Answers	Data Needed	ExamGuard Use
Supervised	What IS this?	Labeled examples	Detect known cheating (phone, gaze, behavior)

Unsupervised	Is this NORMAL?	Only normal examples	Find unknown/new cheating methods
Reinforcement	Should I ACT?	Reward/penalty feedback	Decide when to alert

The key insight:

- Supervised catches what we KNOW about
- Unsupervised catches what we DON'T know about
- Reinforcement makes the final SMART decision

Together, they create a system that is both knowledgeable and adaptive, exactly what ExamGuard needs.

Models Used for Each Type (Summary)

ML Type	Model	Why This Model
Supervised	YOLOv8	Fastest real-time object detector
Supervised	CNN (ResNet/EfficientNet)	Best for image classification
Supervised	CNN + LSTM	CNN sees frames, LSTM remembers sequences
Unsupervised	Autoencoder	Perfect for learning "normal" and flagging "abnormal"
Unsupervised	K-Means	Simple, fast clustering of behavior types
Reinforcement	DQN / PPO	Proven for decision-making with discrete choices

ExamGuard Model Map

The 5 Questions to Choose Any ML Model

Before picking a model, ask these 5 questions:

1. WHAT is the sub-problem? → Break the big problem into small pieces
2. WHAT kind of data do I have? → Images? Numbers? Sequences? Labels?
3. WHAT type of ML fits? → Supervised? Unsupervised? Reinforcement?
4. WHAT models can do this? → List candidate models
5. WHY this specific model? → Speed? Accuracy? Simplicity?

Let's apply these 5 questions to every part of ExamGuard.

ExamGuard Sub-Problems

The big problem "detect cheating in exams" breaks down into 7 sub-problems:

1. Detect prohibited objects (phones, chits, earpieces)
2. Track where students are looking (gaze direction)
3. Classify behavior over time (cheating vs normal)
4. Find unusual/unexpected behavior (anomaly detection)
5. Group similar behaviors (pattern discovery)
6. Decide when to send alerts (smart alerting)
7. Track students across cameras (identity matching)

Each sub-problem needs its own model (or combination of models).

The Full Model Map

#	Sub-Problem	Data Type	ML Type	Model	Why This Model
1	Phone/Object Detection	Images (frames)	Supervised	YOLOv8	Fastest real-time detector. Can find and locate multiple objects in one frame in under 10ms. Other detectors (Faster R-CNN, SSD) are slower.
2	Gaze Direction	Face images	Supervised	CNN (EfficientNet)	Best accuracy-to-speed ratio for image classification. Takes a face crop and predicts gaze direction (left/right/down/center).
3	Behavior Classification	Video clips (sequences of frames)	Supervised	CNN + LSTM	CNN extracts features from each frame, LSTM remembers the sequence over time. Together they understand "what happened over 30 seconds."
4	Anomaly Detection	Normal behavior data	Unsupervised	Autoencoder	Learns to compress and reconstruct normal behavior. Anything it can't reconstruct well = anomaly. No labels needed.

5	Behavior Grouping	Feature vectors	Unsupervised	K-Means Clustering	Simple and fast. Groups students by behavior similarity. Finds clusters of suspicious coordinated behavior.
6	Alert Decision	State + reward signals	Reinforcement	DQN / PPO	Learns the optimal policy for when to alert. DQN for simple decisions, PPO for more complex multi-factor decisions.
7	Person Re-ID	Person images	Supervised	Re-ID CNN (OSNet)	Creates unique appearance signatures. Matches same person across different cameras using appearance features.

Detailed Breakdown: Why Each Model?

1. Phone/Object Detection: YOLOv8

The 5 questions:

- 1. Sub-problem: Find phones, chits, books, earpieces in video frames
- 2. Data: Images (video frames), with labeled bounding boxes
- 3. ML type: Supervised (we have labeled examples of objects)
- 4. Candidates: YOLOv8, Faster R-CNN, SSD, EfficientDet
- 5. Why YOLO: Speed. ExamGuard is real-time. YOLO processes a frame in 5-10ms. Faster R-CNN takes 50-100ms. That 10x difference matters when processing hundreds of cameras.

Input: A video frame [640 x 640 pixels]

Output: Bounding boxes with labels

→ "phone" at position (x:312, y:445) with 89% confidence

→ "paper chit" at position (x:156, y:223) with 72% confidence

2. Gaze Direction: CNN (EfficientNet)

The 5 questions:

- 1. Sub-problem: Determine where a student is looking
- 2. Data: Cropped face images, labeled with gaze direction
- 3. ML type: Supervised (labeled gaze directions)
- 4. Candidates: ResNet, VGG, EfficientNet, MobileNet
- 5. Why EfficientNet: Best accuracy per computation. Smaller than ResNet but equally accurate. Important when running alongside other models.

Input: Cropped face image [224 x 224 pixels]

Output: Gaze class

→ "looking_left" (67%)

→ "looking_at_paper" (20%)

→ "looking_at_neighbor" (13%)

3. Behavior Classification: CNN + LSTM

The 5 questions:

- 1. Sub-problem: Classify a 30-second clip as cheating, suspicious, or normal
- 2. Data: Video clips (sequences of frames) with behavior labels
- 3. ML type: Supervised (labeled video clips)
- 4. Candidates: 3D CNN, CNN+LSTM, Transformer, Two-Stream Network
- 5. Why CNN+LSTM: CNN handles the visual part (what each frame shows), LSTM handles the temporal part (how frames relate over time). This combination is well-proven for action recognition and easier to train than 3D CNNs or Transformers with limited data.

Input: 30-second video clip [90 frames at 3fps]
Frame 1 → CNN → features_1
Frame 2 → CNN → features_2
...
Frame 90 → CNN → features_90

[features_1, features_2, ..., features_90] → LSTM

Output: Behavior class
→ "suspicious_looking" (74%)
→ "normal_writing" (18%)
→ "passing_notes" (8%)

4. Anomaly Detection: Autoencoder

The 5 questions:

- 1. Sub-problem: Find any behavior that's unusual/unexpected
- 2. Data: Lots of normal behavior clips (NO labels needed)
- 3. ML type: Unsupervised (learn normal, flag abnormal)
- 4. Candidates: Autoencoder, Isolation Forest, One-Class SVM, GAN
- 5. Why Autoencoder: Intuitive and effective for video. Learns to compress and reconstruct normal behavior. When it sees abnormal behavior, reconstruction error is high = anomaly detected. Works well with visual data unlike Isolation Forest.

TRAINING (only on normal behavior):
Normal clip → [Encoder] → compressed → [Decoder] → reconstructed clip
Loss = difference between original and reconstructed
Model learns to perfectly reconstruct normal behavior

DETECTION:
Normal clip → reconstruct → low error (0.05) → NORMAL
Weird clip → reconstruct → high error (0.73) → ANOMALY!

5. Behavior Grouping: K-Means

The 5 questions:

- 1. Sub-problem: Group students by behavior patterns
- 2. Data: Feature vectors extracted from behavior analysis
- 3. ML type: Unsupervised (find natural groups)
- 4. Candidates: K-Means, DBSCAN, Hierarchical Clustering
- 5. Why K-Means: Simplest and fastest. When you need quick grouping of behavior patterns during a live exam, speed matters more than finding perfect cluster shapes.

Input: Behavior features for 100 students
    Student 1: [0.1, 0.9, 0.2, 0.8, ...] (lots of head movement)
    Student 2: [0.8, 0.1, 0.9, 0.1, ...] (steady writing)
    ...

Output: Clusters
    Cluster A (85 students): Steady writers → NORMAL
    Cluster B (10 students): Occasional lookers → MONITOR
    Cluster C (5 students): Unusual movement → INVESTIGATE

6. Alert Decision: DQN / PPO

The 5 questions:

- 1. Sub-problem: Decide when to alert and when to wait
- 2. Data: State (suspicion scores) + Reward (invigilator feedback)
- 3. ML type: Reinforcement Learning (learn from outcomes)
- 4. Candidates: DQN, PPO, A3C, SAC
- 5. Why DQN/PPO: DQN is simpler and works well for discrete decisions (alert / don't alert). PPO is more stable for complex decisions (alert level: low / medium / high / critical).

State: [phone_score: 0.73, gaze_score: 0.68, anomaly_score: 0.82,
    time_since_last_alert: 120sec, hall_alert_count: 5]

Action: ALERT (with confidence: high)

Reward: Invigilator confirms → +100
  OR   Invigilator dismisses → -50

7. Person Re-ID: OSNet

The 5 questions:

- 1. Sub-problem: Match same person across different cameras
- 2. Data: Person images from different camera angles with identity labels
- 3. ML type: Supervised (labeled person identities)
- 4. Candidates: OSNet, ResNet-based Re-ID, BoT, TransReID
- 5. Why OSNet: Designed specifically for Re-ID. Lightweight enough for real-time use. Good accuracy even with low resolution images from surveillance cameras.

Camera 1: [Person crop] → OSNet → vector [0.23, 0.87, 0.12, ...]
Camera 3: [Person crop] → OSNet → vector [0.25, 0.85, 0.14, ...]

Distance between vectors: 0.04 → Very close → SAME PERSON

Camera 2: [Different person] → OSNet → vector [0.91, 0.13, 0.78, ...]
Distance from first person: 0.82 → Far apart → DIFFERENT PERSON

Expert Tricks for Model Selection

1. Transfer Learning: Don't Train From Scratch

BAD: Train YOLO from zero on your exam data
    Needs: 100,000+ labeled images

Time: Weeks of training

GOOD: Start with YOLO pre-trained on COCO dataset (80 object types)
Fine-tune on your exam data
Needs: 1,000-5,000 labeled images
Time: Hours of training

Transfer learning means: take a model that already knows how to see (trained on millions of images) and teach it to see YOUR specific things (phones in exam halls). It's like hiring an experienced security guard instead of training a baby.

2. Start MVP (Minimum Viable Product)

Phase 1: Just phone detection with YOLO → Does it work at all?
Phase 2: Add gaze tracking with CNN → Can it see where people look?
Phase 3: Add behavior analysis with LSTM → Can it understand sequences?
Phase 4: Add anomaly detection with Autoencoder → Can it catch unexpected things?
Phase 5: Add RL for smart alerting → Can it make good decisions?

DON'T try to build everything at once!

3. Test Multiple Models

For object detection, try:
YOLOv8-nano → fastest, less accurate
YOLOv8-small → good balance
YOLOv8-medium → more accurate, slower

Measure on YOUR exam hall data and pick the best trade-off.

4. Data Size Rules of Thumb

Model	Minimum Data	Good Performance	Production
YOLO (fine-tune)	500 images	2,000 images	5,000+ images
CNN (fine-tune)	500 images	2,000 images	5,000+ images
CNN + LSTM	1,000 clips	5,000 clips	10,000+ clips
Autoencoder	2,000 normal clips	5,000 normal clips	10,000+ normal clips
RL Agent	100 exam sessions	500 sessions	1,000+ sessions

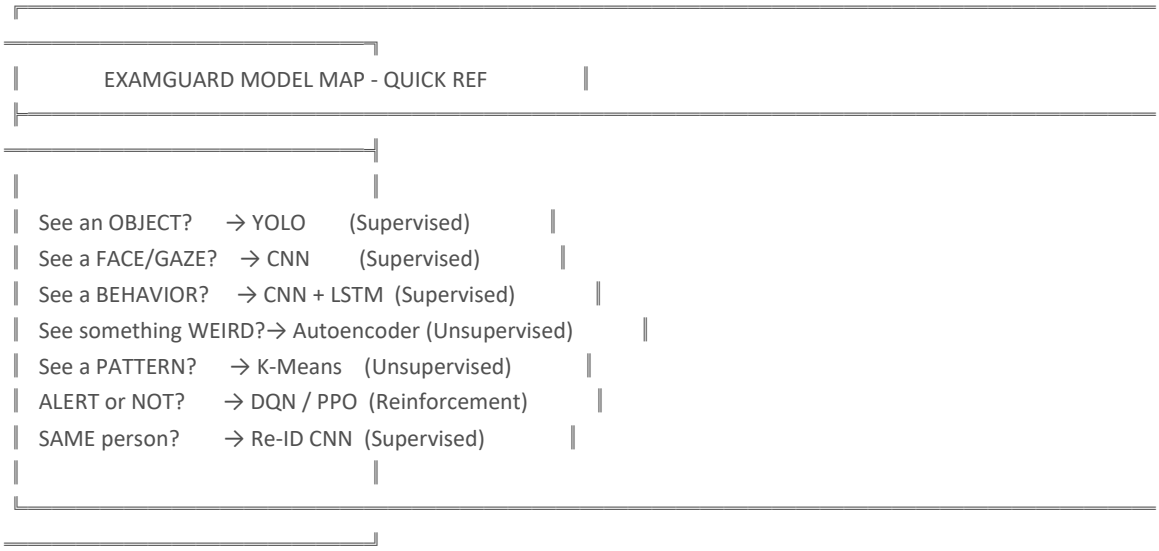
5. Real-Time Constraint

ExamGuard must process each frame in under 100ms (10 FPS minimum)

Model speed budget per frame:
YOLO detection: ~10ms
Face/Gaze CNN: ~15ms
Behavior LSTM: ~20ms
Autoencoder: ~10ms
RL decision: ~5ms
Other processing: ~10ms
———————————
Total: ~70ms ✓ (under 100ms budget)

If a model is too slow, you must either use a smaller version or optimize it.

Quick Reference Card



ExamGuard Overview - Resources

Search Terms for Research

When researching ExamGuard-related topics, use these search terms to find relevant information:

System Design Research

- "exam proctoring AI system" - Find how existing companies solve this problem
- "AI surveillance system architecture" - Understand how large-scale AI monitoring systems are designed
- "real-time video analytics architecture" - Learn about processing live video feeds with AI
- "multi-camera monitoring system design" - Study how to handle many cameras simultaneously
- "AI cheating detection in exams" - Find academic papers and articles on this specific topic

Technical Research

- "YOLO object detection real-time" - The core detection model we'll use
- "gaze estimation deep learning" - How AI tracks where people look
- "video anomaly detection autoencoder" - Finding unusual behavior in video
- "reinforcement learning alert system" - Smart decision-making for alerts
- "person re-identification deep learning" - Tracking people across cameras

Similar Products to Study

Understanding existing products helps you design ExamGuard better. Study what they do well and what they miss.

1. Proctorio

- What it does: Online exam proctoring (monitors students taking exams on their computers)
- How it works: Uses the student's webcam and microphone to monitor during online exams
- What it detects: Eye movement, head movement, background noise, browser tab switching, other people in the room
- Limitation: Only works for online exams (not physical exam halls like ExamGuard)
- What to learn from it: Their gaze tracking and behavior analysis approaches
- Website: proctorio.com

2. ExamSoft (now part of Turnitin)

- What it does: Secure exam platform with AI monitoring
- How it works: Locks down the computer during exams and records video for review
- What it detects: Facial recognition (is it the right student?), eye tracking, audio analysis
- Limitation: Post-exam review (flags clips for human review AFTER the exam, not real-time)
- What to learn from it: Their flagging system and how they handle false positives
- Website: examsoft.com

3. Respondus LockDown Browser + Monitor

- What it does: Locks the browser during online exams and records webcam
- How it works: Prevents students from opening other programs; webcam recording analyzed by AI

- What it detects: Missing student, multiple people, looking away, speaking
- Limitation: Online only, not real-time (analysis happens after exam)
- What to learn from it: Simple but effective detection rules; their approach to flagging severity levels
- Website: respondus.com

4. Honorlock

- What it does: AI-powered online proctoring with live proctors available
- How it works: AI monitors the exam in real-time; if something suspicious happens, a live proctor can intervene
- What it detects: Phone detection (searches for exam questions), secondary device detection, voice detection
- Limitation: Online exams only
- What to learn from it: Their hybrid AI + human approach is very similar to ExamGuard's philosophy
- Website: honorlock.com

Comparison with ExamGuard:

Feature	Proctorio	ExamSoft	Respondus	Honorlock	ExamGuard
Environment	Online	Online	Online	Online	Physical hall
Real-time?	Yes	No	No	Yes	Yes
Multi-camera?	No (1 webcam)	No	No	No	Yes (5+ cameras)
Object detection?	Limited	No	No	Phone only	Yes (multiple objects)
Behavior analysis?	Basic	Basic	Basic	Moderate	Advanced (LSTM)
Anomaly detection?	No	No	No	No	Yes (Autoencoder)
Human in the loop?	Optional	Post-exam	Post-exam	Yes	Yes (always)

Key insight: All existing products focus on ONLINE exams. ExamGuard fills a gap by addressing PHYSICAL exam halls with multi-camera coverage. This is our unique value.

YouTube Channels and Videos

System Design

- Search: "AI surveillance system design tutorial"
- Search: "real-time object detection system architecture"
- Search: "building a video analytics pipeline"

Similar Project Walkthroughs

- Search: "exam cheating detection using deep learning"
- Search: "YOLO object detection project tutorial"
- Search: "behavior recognition using CNN LSTM"
- Search: "anomaly detection in surveillance video"

Understanding the Problem Space

- Search: "how AI proctoring works"

- Search: "problems with AI exam monitoring" (learn from criticisms to build better)
- Search: "ethics of AI surveillance in education"

Academic Papers (for later)

When you're more advanced, these papers will be valuable:

- 1. "A Survey on Deep Learning Based Video Anomaly Detection" - Comprehensive overview of anomaly detection methods
- 2. "YOLO: Real-Time Object Detection" - The original YOLO paper series
- 3. "Deep Learning for Gaze Estimation" - Methods for tracking eye direction
- 4. "Person Re-identification: Past, Present and Future" - Multi-camera tracking methods

Search for these on Google Scholar (scholar.google.com) or Papers With Code (paperswithcode.com).

Recommended Study Order

Week 1: Read about Proctorio and Honorlock
→ Understand what's already been done

Week 2: Watch YouTube videos on YOLO and object detection
→ See how the core technology works

Week 3: Read about system architecture for video analytics
→ Understand how to connect everything

Week 4: Study ethics of AI surveillance
→ Understand the responsibility of building such a system

Key Questions to Ask While Researching

As you study these resources, keep asking:

- 1. What works well in existing systems? (Copy the good ideas)
- 2. What fails in existing systems? (Avoid their mistakes)
- 3. What's missing from existing systems? (That's where ExamGuard adds value)
- 4. What are the ethical concerns? (Build responsibly)
- 5. What would make an invigilator's life easier? (Design for the user)